

OBJECT ORIENTED PROGRAMMING WITH C++

Unit-IV: Operator Overloading and Inheritance

Comprehensive Study Notes

UNIT-IV: OPERATOR OVERLOADING AND INHERITANCE

1. Operator Overloading

1.1 Introduction to Operator Overloading

Operator overloading allows us to redefine the way operators work for user-defined data types (classes). It gives special meaning to existing operators.

1.2 The `operator` Keyword

The `operator` keyword is used to overload operators in C++.

Syntax:

```
return_type operator symbol (arguments) {  
    // function body  
}
```

Example:

```
class Complex {  
    float real, imag;  
public:  
    Complex(float r = 0, float i = 0) { real = r; imag = i; }  
    Complex operator + (Complex c) {  
        Complex temp;  
        temp.real = real + c.real;  
        temp.imag = imag + c.imag;  
        return temp;  
    }  
    void display() {  
        cout << real << " + " << imag << "i" << endl;  
    }  
};
```

```
int main() {  
    Complex c1(3.5, 2.5), c2(1.5, 4.5), c3;  
    c3 = c1 + c2;  
    c1.display();  
    c2.display();  
    c3.display();  
    return 0;  
}
```

Output:

```
3.5 + 2.5i  
1.5 + 4.5i  
5 + 7i
```

1.3 Overloading Unary Operators

Unary operators like `++`, `--`, `-` operate on a single operand.

Overloading Unary Minus (-)

```
class Number {  
    int value;  
public:  
    Number(int v = 0) { value = v; }  
    Number operator - () {  
        Number temp;  
        temp.value = -value;  
        return temp;  
    }  
    void display() { cout << "value: " << value << endl; }  
};  
  
int main() {  
    Number n1(10), n2;  
    n2 = -n1;  
    cout << "n1: "; n1.display();  
    cout << "n2: "; n2.display();  
    return 0;  
}
```

Output:

```
n1: 10  
n2: -10
```

Overloading Prefix and Postfix Increment (++)

```
class Counter {  
    int count;  
public:  
    Counter(int c = 0) { count = c; }  
    // Prefix increment  
    Counter operator ++ () {  
        ++count;  
        return *this;  
    }  
    // Postfix increment (int parameter distinguishes)  
    Counter operator ++ (int) {  
        Counter temp = *this;  
        count++;  
        return temp;  
    }  
    void display() { cout << "count: " << count << endl; }  
};  
  
int main() {  
    Counter c1(5), c2, c3;  
    c2 = ++c1; // Prefix: c1 becomes 6, c2 = 6  
    c3 = c1++; // Postfix: c3 = 6, c1 becomes 7  
    cout << "c1: "; c1.display();  
    cout << "c2: "; c2.display();  
    cout << "c3: "; c3.display();  
    return 0;  
}
```

Output:

```
c1: 7  
c2: 6  
c3: 6
```

1.4 Overloading Binary Operators

Binary operators like `+`, `-`, `*` operate on two operands.

Overloading Addition (+), Subtraction (-), and Multiplication (*)

```
class Complex {
    float real, imag;
public:
    Complex(float r = 0, float i = 0) { real = r; imag = i; }
    Complex operator + (Complex c) {
        return Complex(real + c.real, imag + c.imag);
    }
    Complex operator - (Complex c) {
        return Complex(real - c.real, imag - c.imag);
    }
    Complex operator * (Complex c) {
        return Complex(real * c.real - imag * c.imag,
                        real * c.imag + imag * c.real);
    }
    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(3, 4), c2(1, 2);
    cout << "c1: "; c1.display();
    cout << "c2: "; c2.display();
    Complex c3 = c1 + c2;
    cout << "c1 + c2: "; c3.display();
    Complex c4 = c1 - c2;
    cout << "c1 - c2: "; c4.display();
    Complex c5 = c1 * c2;
    cout << "c1 * c2: "; c5.display();
    return 0;
}
```

Output:

```
c1: 3 + 4i
c2: 1 + 2i
c1 + c2: 4 + 6i
c1 - c2: 2 + 2i
c1 * c2: -5 + 10i
```

1.5 Operator Arguments

Operators can be overloaded as:

1. **Member Functions** (one argument for binary, zero for unary)
2. **Friend Functions** (two arguments for binary, one for unary)

Friend Function Example:

```
class Complex {
    float real, imag;
public:
    Complex(float r = 0, float i = 0) { real = r; imag = i; }
    friend Complex operator + (Complex c1, Complex c2);
    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

// Friend function definition
Complex operator + (Complex c1, Complex c2) {
    return Complex(c1.real + c2.real, c1.imag + c2.imag);
}

int main() {
    Complex c1(3, 4), c2(1, 2);
    Complex c3 = c1 + c2;
    c3.display();
    return 0;
}
```

1.6 Operator Return Values

Operators can return:

1. **Objects** (like +, -, *)
2. **References** (like = for chaining)
3. **Void** (for output operators)

```
class MyClass {
    int value;
public:
    MyClass(int v = 0) { value = v; }
    MyClass operator + (MyClass obj) {
```

```

        return MyClass(value + obj.value);
    }
    MyClass& operator = (MyClass obj) {
        value = obj.value;
        return *this;
    }
    void display() { cout << "Value: " << value << endl; }
};

int main() {
    MyClass a(10), b(20), c, d;
    c = a + b;
    cout << "a + b: "; c.display();
    d = c = a;
    cout << "After chaining: "; d.display();
    return 0;
}

```

Output:

```

a + b: 30
After chaining: 40

```

1.7 Overloading Comparison Operators

```

class Time {
    int hours, minutes;
public:
    Time(int h = 0, int m = 0) { hours = h; minutes = m; }
    bool operator > (Time t) {
        if (hours > t.hours) return true;
        if (hours == t.hours && minutes > t.minutes) return true;
        return false;
    }
    bool operator == (Time t) {
        return (hours == t.hours && minutes == t.minutes);
    }
    void display() { cout << hours << ":" << minutes << endl; }
};

int main() {
    Time t1(10, 30), t2(11, 15);
    if (t1 > t2) cout << "t1 is greater" << endl;
}

```

```
    else cout << "t2 is greater" << endl;
    return 0;
}
```

1.8 Nameless Temporary Objects

Nameless temporary objects are created during expression evaluation.

```
class Number {
    int value;
public:
    Number(int v = 0) { value = v; }
    Number operator + (Number obj) {
        return Number(value + obj.value); // nameless temporary object
    }
    void display() { cout << "Value: " << value << endl; }
};

int main() {
    Number a(10), b(20);
    Number c = a + b; // temporary object created and assigned
    c.display();
    return 0;
}
```

1.9 Overloading Input/Output Operators

```
class Complex {
    float real, imag;
public:
    Complex(float r = 0, float i = 0) { real = r; imag = i; }
    friend ostream& operator << (ostream& out, Complex c);
    friend istream& operator >> (istream& in, Complex& c);
};

ostream& operator << (ostream& out, Complex c) {
    out << c.real << " + " << c.imag << "i";
    return out;
}

istream& operator >> (istream& in, Complex& c) {
    cout << "Enter real part: ";
```



```
        in >> c.real;
        cout << "Enter imaginary part: ";
        in >> c.imag;
        return in;
    }

    int main() {
        Complex c1, c2;
        cin >> c1;
        cin >> c2;
        cout << "c1 = " << c1 << endl;
        cout << "c2 = " << c2 << endl;
        return 0;
    }
```

Input:

```
Enter first complex number:
Enter real part: 3
Enter imaginary part: 4
Enter second complex number:
Enter real part: 1
Enter imaginary part: 2
```

Output:

```
c1 = 3 + 4i
c2 = 1 + 2i
c1 + c2 = 4 + 6i
```

1.10 Data Conversion

Data conversion involves converting between different data types.

Converting Basic Type to Class Type

```
class Distance {
    int meters, centimeters;
public:
    Distance(int m = 0, int cm = 0) { meters = m; centimeters = cm; }
    Distance(int totalCm) { // Basic to class type
        meters = totalCm / 100;
    }
}
```

```
        centimeters = totalCm % 100;
    }
    void display() { cout << meters << "m " << centimeters << "cm" << endl; }
};

int main() {
    Distance d1 = 150; // int to Distance
    Distance d2 = 250;
    Distance d3 = 300;
    d1.display();
    d2.display();
    d3.display();
    return 0;
}
```

Output:

```
1m 50cm
2m 50cm
3m 0cm
```

Converting Class Type to Basic Type

```
class Distance {
    int meters, centimeters;
public:
    Distance(int m = 0, int cm = 0) { meters = m; centimeters = cm; }
    operator int() { // Class to basic type conversion
        return meters * 100 + centimeters;
    }
    void display() { cout << meters << "m " << centimeters << "cm" << endl; }
};

int main() {
    Distance d(2, 50);
    d.display();
    int totalCm = d; // Distance to int
    cout << "Total centimeters: " << totalCm << "cm" << endl;
    return 0;
}
```

Output:

```
2m 50cm  
Total centimeters: 250cm
```

Converting Between Class Types

```
class Inches {  
    int inches;  
public:  
    Inches(int i = 0) { inches = i; }  
    int getInches() { return inches; }  
    void display() { cout << "Inches: " << inches << " inches" << endl; }  
};  
  
class Feet {  
    int feet;  
public:  
    Feet(int f = 0) { feet = f; }  
    Feet(Inches i) { // Inches to Feet conversion  
        feet = i.getInches() / 12;  
    }  
    void display() { cout << "Feet: " << feet << " feet" << endl; }  
};  
  
int main() {  
    Feet f(5);  
    Inches i(60);  
    f = i; // Inches to Feet  
    f.display();  
    i.display();  
    return 0;  
}
```

Output:

```
Feet: 5 feet  
Inches: 60 inches
```

1.11 Pitfalls of Operator Overloading

Pitfall	Description
---------	-------------

Ambiguity	Multiple meanings of the same operator
Loss of Meaning	Using operators in unintended ways
Complexity	Overloading too many operators makes code complex
Performance	Can be slower than regular functions
Limitations	Cannot overload operators for built-in types
No New Operators	Cannot create new operators

Rules for Operator Overloading:

Rule	Description
1	Cannot overload operators for built-in types
2	Cannot create new operators
3	Cannot change precedence and associativity
4	Must have at least one user-defined type
5	Cannot overload: :: , .* , . , ?

2. Inheritance

2.1 Introduction to Inheritance

Inheritance is the process by which one class (derived class) acquires the properties and methods of another class (base class).

2.2 Defining a Derived Class

Syntax:

```
class derived_class_name : access_specifier base_class_name {  
    // additional members  
};
```

Example:

```
#include <iostream>  
#include <string>  
using namespace std;  
  
// Base class  
class Animal {  
protected:  
    string name;  
    int age;  
public:  
    void setData(string n, int a) { name = n; age = a; }  
    void eat() { cout << name << " is eating." << endl; }  
    void sleep() { cout << name << " is sleeping." << endl; }  
};  
  
// Derived class  
class Dog : public Animal {  
public:  
    void bark() { cout << name << " is barking: Woof! Woof!" << endl; }  
};  
  
class Cat : public Animal {  
public:
```

```

        void meow() { cout << name << " is meowing: Meow! Meow!" << endl; }
    };

int main() {
    Dog d;
    d.setData("Tommy", 3);
    cout << "Name: " << "Tommy, Age: 3" << endl;
    cout << "Breed: German Shepherd" << endl;
    d.eat();
    d.bark();

    Cat c;
    c.setData("Kitty", 2);
    cout << "Name: Kitty, Age: 2" << endl;
    c.sleep();
    c.meow();
    return 0;
}

```

Output:

```

Name: Tommy, Age: 3
Breed: German Shepherd
Tommy is eating.
Tommy is barking: Woof! Woof!
Name: Kitty, Age: 2
Kitty is sleeping.
Kitty is meowing: Meow! Meow!

```

2.3 Access Specifiers in Inheritance

Base Class	Derived Class	Access to Base Members
public	public	public members become public
public	protected	public members become protected
public	private	public members become private
protected	public	protected members become protected
protected	protected	protected members become protected

protected	private	protected members become private
private	any	private members not inherited

```

class Base {
private:
    int a;
protected:
    int b;
public:
    int c;
};

class Derived : public Base {
    // a is not accessible
    // b is protected
    // c is public
};

```

2.4 Derived Class Constructors

The derived class constructor calls the base class constructor first.

```

class Base {
public:
    Base(int x) {
        cout << "Base constructor called with " << x << endl;
    }
    ~Base() { cout << "Base destructor called" << endl; }
};

class Derived : public Base {
public:
    Derived(int x, int y) : Base(x) {
        cout << "Derived constructor called with " << y << endl;
    }
    ~Derived() { cout << "Derived destructor called" << endl; }
};

int main() {
    Derived d(10, 20);
}

```

```
    return 0;
}
```

Output:

```
Base constructor called with 10
Derived constructor called with 20
Derived destructor called
Base destructor called
```

2.5 Overriding Member Functions

Function overriding occurs when a derived class redefines a member function of the base class.

```
class Base {
public:
    void display() { cout << "Base class display" << endl; }
    virtual void show() { cout << "Base class show" << endl; }
};

class Derived : public Base {
public:
    void display() { cout << "Derived class display" << endl; }
    void show() override { cout << "Derived class show" << endl; }
};

int main() {
    Base b;
    Derived d;
    b.display();
    d.display();
    b.show();
    d.show();
    return 0;
}
```

Output:

```
Base class display
Derived class display
```


Base class show
Derived class show

2.6 Class Hierarchies

```
class GrandParent {
public:
    GrandParent() { cout << "GrandParent constructor" << endl; }
};

class Parent : public GrandParent {
public:
    Parent() { cout << "Parent constructor" << endl; }
};

class Child : public Parent {
    string name;
    int age;
    string school;
public:
    Child(string n, int a, string s) {
        name = n;
        age = a;
        school = s;
        cout << "Child constructor" << endl;
    }
    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "School: " << school << endl;
    }
};

int main() {
    Child c("Rahul", 15, "St. Mary's School");
    c.display();
    return 0;
}
```

Output:

GrandParent constructor
Parent constructor

Child constructor
Name: Rahul
Age: 15
School: St. Mary's School

2.7 Public and Private Inheritance

Public Inheritance

```
class Base {  
public:  
    int pub;  
protected:  
    int prot;  
private:  
    int priv;  
};  
  
class DerivedPublic : public Base {  
    // pub remains public  
    // prot remains protected  
    // priv is not accessible  
};
```

Private Inheritance

```
class DerivedPrivate : private Base {  
    // pub becomes private  
    // prot becomes private  
    // priv is not accessible  
};
```

2.8 Levels of Inheritance

Type	Description
Single Inheritance	One base class, one derived class
Multilevel Inheritance	Chain of classes (A -> B -> C)
Hierarchical Inheritance	One base, multiple derived classes

Multiple Inheritance	Multiple base classes
Hybrid Inheritance	Combination of multiple types

2.9 Multiple Inheritance

A class can inherit from **multiple base classes**.

Syntax:

```
class Derived : access_specifier Base1, access_specifier Base2 {  
    // additional members  
};
```

Example:

```
class Father {  
protected:  
    string fatherName;  
public:  
    Father(string n) { fatherName = n; }  
};  
  
class Mother {  
protected:  
    string motherName;  
public:  
    Mother(string n) { motherName = n; }  
};  
  
class Child : public Father, public Mother {  
    string childName;  
public:  
    Child(string fn, string mn, string cn) : Father(fn), Mother(mn) {  
        childName = cn;  
    }  
    void display() {  
        cout << "Father: " << fatherName << endl;  
        cout << "Mother: " << motherName << endl;  
        cout << "Child: " << childName << endl;  
    }  
};
```

```
int main() {  
    Child c("Mr. Sharma", "Mrs. Sharma", "Rahul");  
    c.display();  
    return 0;  
}
```

Output:

```
Father: Mr. Sharma  
Mother: Mrs. Sharma  
Child: Rahul
```

2.10 Ambiguity in Multiple Inheritance

When the same member name exists in multiple base classes, ambiguity occurs.

Example: Ambiguity

```
class Base1 {  
public:  
    void show() { cout << "Base1 show" << endl; }  
};  
  
class Base2 {  
public:  
    void show() { cout << "Base2 show" << endl; }  
};  
  
class Derived : public Base1, public Base2 {  
    // Ambiguity: which show() to call?  
};
```

Resolving Ambiguity:

Method	Syntax	Description
Scope Resolution	<code>object.BaseClass::function()</code>	Calls specific base class function

Overriding	Define function in derived class	Overrides both base class functions
Virtual Inheritance	class Derived : virtual Base	Avoids duplicate copies

2.11 Aggregation - Classes Within Classes

Aggregation is the process of creating objects of one class inside another class.

```
#include <iostream>
#include <string>
using namespace std;

class Address {
    string city;
    string state;
    int pincode;
public:
    Address(string c, string s, int p) { city = c; state = s; pincode = p; }
    void display() {
        cout << "City: " << city << endl;
        cout << "State: " << state << endl;
        cout << "Pincode: " << pincode << endl;
    }
};

class Student {
    int rollNo;
    string name;
    Address addr;
public:
    Student(int r, string n, Address a) : addr(a) {
        rollNo = r;
        name = n;
    }
    void display() {
        cout << "Roll No: " << rollNo << endl;
        cout << "Name: " << name << endl;
        cout << "Address: " << endl;
        addr.display();
    }
};
```

```
int main() {  
    Address a("Shimla", "Himachal Pradesh", 171001);  
    Student s(101, "Rahul", a);  
    s.display();  
    return 0;  
}
```

Output:

```
Roll No: 101  
Name: Rahul  
Address:  
City: Shimla  
State: Himachal Pradesh  
Pincode: 171001
```

Questions

Short Answer Questions (2-5 Marks)

1. What is operator overloading?
2. What is the `operator` keyword in C++?
3. What are the rules for operator overloading?
4. What is the difference between prefix and postfix increment?
5. What is inheritance in C++?
6. What is the difference between public and private inheritance?
7. What is function overriding?
8. What is multiple inheritance?
9. What is ambiguity in inheritance and how is it resolved?
10. What is aggregation in C++?

Long Answer Questions (10 Marks)

1. Explain operator overloading with examples.
2. Explain unary and binary operator overloading.
3. Explain inheritance and its types with examples.
4. Explain multiple inheritance and ambiguity resolution.
5. Explain data conversion in C++ with examples.

Program Writing Questions (5 Marks)

1. Write a program to overload `+` and `-` operators for complex numbers.
2. Write a program to demonstrate inheritance with a base class `Employee` and derived classes `Manager` and `Clerk`.
3. Write a program to overload prefix and postfix increment operators.
4. Write a program to demonstrate multiple inheritance.
5. Write a program to overload input/output operators.

Objective Questions (1 Mark)

1. The `operator` keyword is used for: (a) Function overloading (b) Operator overloading **(b)** (c) Both (d) None
2. Which operator cannot be overloaded? (a) + (b) - (c) . **(c)** (d) *
3. Inheritance is: (a) Code reusability **(a)** (b) Data hiding (c) Polymorphism (d) None
4. Multiple inheritance means: (a) One base, multiple derived (b) Multiple base, one derived **(b)** (c) Both (d) None
5. Ambiguity in inheritance is resolved using: (a) Scope resolution **(a)** (b) Operator overloading (c) Both (d) None

QUICK REVISION

Overloadable Operators

```
+ - * / % ++ -- == != < >
<= >= && || ! = += -= *= /=
& | ^ ~ << >> [] () -> new delete
```

Inheritance Types

Type	Description
Single	One base, one derived
Multilevel	A -> B -> C
Multiple	Multiple base classes
Hierarchical	One base, multiple derived
Hybrid	Combination of types

Key Points

Feature	Description
Operator Overloading	Redefine operators for user-defined types
Inheritance	Acquiring properties of base class
Function Overriding	Redefining base class function in derived class
Ambiguity	Conflict in multiple inheritance
Aggregation	Classes within classes

End of Unit-IV Notes